Block Color Sorting

1. Function Description

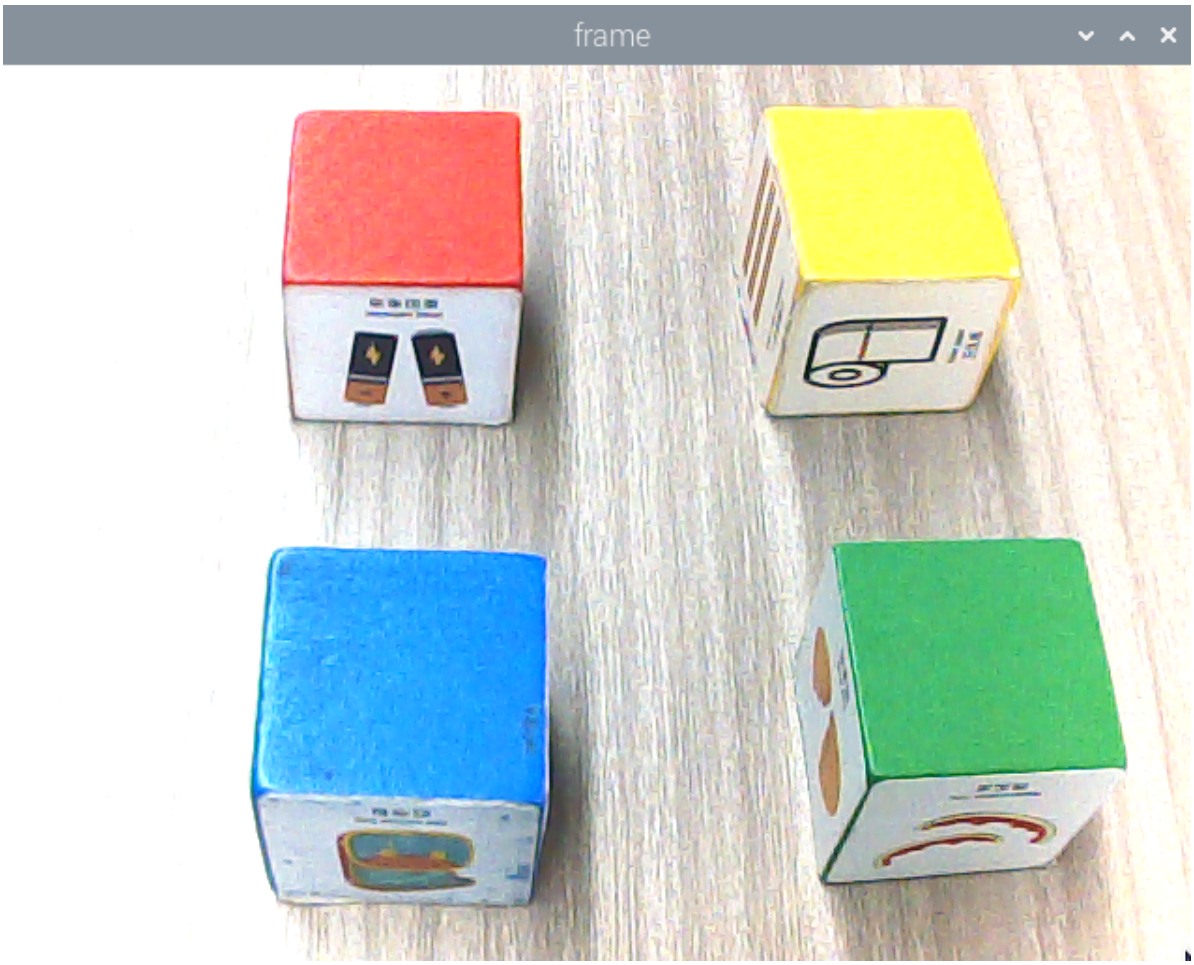
After the program runs, the robotic arm will move to its initial position, then place 3cm high color blocks of different colors within the camera's view. Press keys to select the color to sort (r-red, g-green, b-blue, y-yellow), then press the spacebar. The robotic arm will grasp the specified color block according to the selected color and place it at the designated position.

2. Function Startup

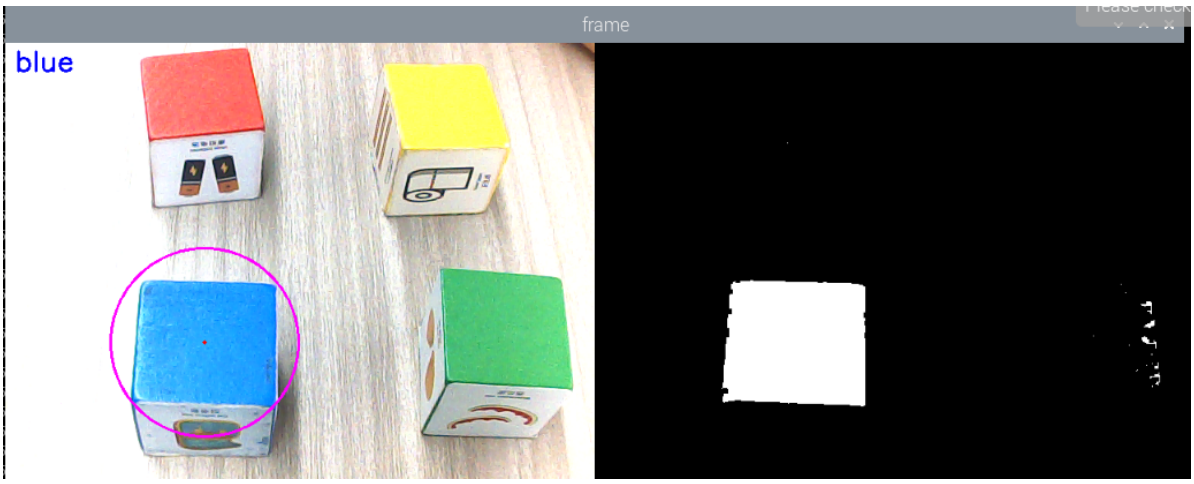
After entering the docker container, enter the following commands in the docker terminal:

```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start color block recognition node
ros2 run dofbot_sorting color_sorting
# Start grasping node
ros2 run dofbot_sorting grasp
```

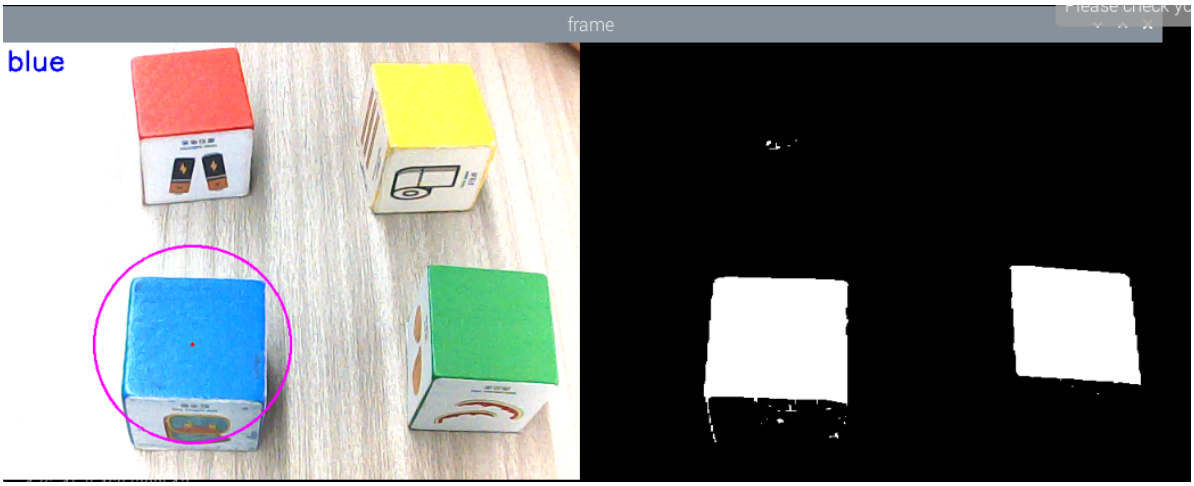
After successfully running the program, the robotic arm will move to its initial position, then place 3cm high color blocks within the camera's view as shown in the figure below.



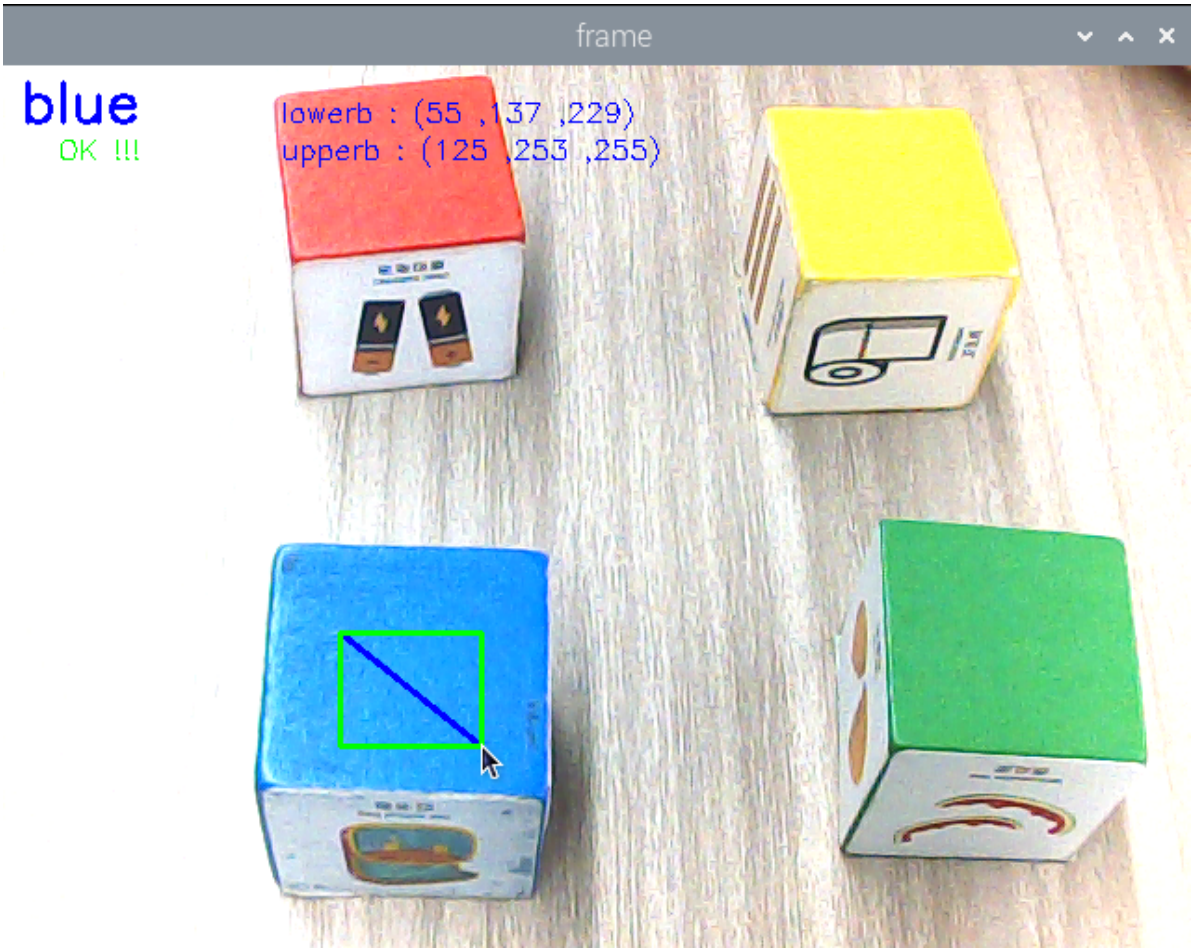
Press r/g/b/y to select the color to sort. If the binary image only shows the target color block, press the spacebar to start sorting. If the binary image cannot display the target color block or mistakenly recognizes other color blocks, press c and then use the mouse to select an area of the target color block to complete color calibration, then press the spacebar to sort. For example, if I press b to select sorting blue blocks, the ideal binary image is shown below:



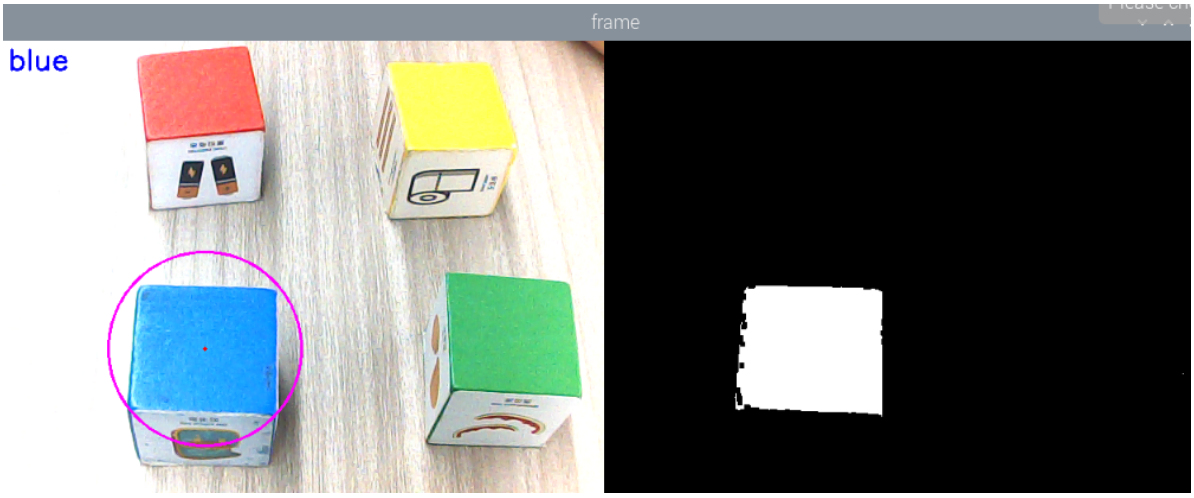
Then press the spacebar, and the robotic arm will grasp the blue block and place it in the designated area. If the recognition effect is not good, for example, if other color blocks appear in the binary image on the right as shown below:



Then press the c key to enter color calibration mode, then use the mouse to select a partial area of the blue block as shown below:



Release the mouse to complete calibration. The calibrated HSV values of the blue block will also be written to the file simultaneously. As shown below, you can press the spacebar to start the grasping and placing action:



3. Code Analysis

3.1. color_sorting

After entering the docker container, the source code path is located at:

```
/root/dofbot_sorting/dofbot_sorting/color_sorting.py
```

Import necessary libraries:

```
import cv2
```

```
import os
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
# Defined color recognition library, source code location is at
/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/color_common.py
from dofbot_sorting.color_common import *
from dofbot_interface.srv import Kinemarics
from dofbot_interface.msg import *
from std_msgs.msg import Bool,Int16
import time
from rclpy.node import Node
import rclpy
import threading
from Arm_Lib import Arm_Device
```

Create robotic arm control object and control the robotic arm to move to initial position:

```
self.Arm = Arm_Device()
self.init_joints = [90, 120, 0, 0, 90, 0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Create color recognition object:

```
self.color = color_detect()
```

Load HSV file paths for red, green, blue, and yellow colors:

```
package_pwd =  "/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/"
self.red_hsv_text = os.path.join(package_pwd, 'red_colorHSV.text')
self.green_hsv_text = os.path.join(package_pwd, 'green_colorHSV.text')
self.blue_hsv_text = os.path.join(package_pwd, 'blue_colorHSV.text')
self.yellow_hsv_text = os.path.join(package_pwd, 'yellow_colorHSV.text')
#HSV content is as follows (using blue_colorHSV.text as example):
#55, 163, 235, 125, 253, 255
#Represent Hmin, Smin, Vmin, Hmax, Smax, Vmax respectively
```

Create publishers and subscribers:

```
# Create subscriber to subscribe to grasp completion topic messages
self.sub_grasp_status =
self.create_subscription(Bool,"grasp_done",self.get_graspStatusCallBack,100)
# Create subscriber to subscribe to image topic messages
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
# Create publisher to publish recognized position information topic messages
self.pos_info_pub = self.create_publisher(AprilTagInfo,"PosInfo",1)
# Create publisher to publish calculated joint 5 angle reference value topic
messages
self.pub_joint5 = self.create_publisher(Int16, "set_joint5", 10)
```

Grasp completion topic callback function get_graspStatusCallBack:

```
# Process grasp completion topic messages. When the received message is True, it indicates the current sorting is completed and the next sorting can be performed. Change self.pub_pos_flag to indicate that the next color block position information can be published
def get_graspStatusCallBack(self,msg):
    if msg.data == True:
        time.sleep(2)
        self.pub_pos_flag = True
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_frame):
    # Process received image topic messages, convert to image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'rgb8')
    rgb_image = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)
    result_image = np.copy(rgb_image)
    key = cv2.waitKey(10)& 0xFF
    # Pass image and key detection results to process function
    result_frame, binary = self.process(rgb_image,key)
    # Create thread to display image
    show_frame = threading.Thread(target=self.img_out, args=
(result_frame,binary,))
    show_frame.start()
    show_frame.join()
    # Detect key press. If spacebar is pressed (spacebar value is 32), modify self.pubPos_flag value to True, indicating that target color block position information topic messages can be published
    if key == 32:
        self.pub_pos_flag = True
    # If target color block exists, determine by checking if the minimum enclosing circle center coordinates are zero, also need to check the minimum enclosing circle radius to filter out some smaller points
    if self.cx!=0 and self.cy!=0 and self.circle_r>30:
        cx = int(self.cx)
        cy = int(self.cy)
        # Based on minimum enclosing circle center coordinates, convert coordinates to normalized coefficients a and b. a is x-axis offset coefficient, b is y-axis offset coefficient. The calculated ba values are the center point coordinates in the world coordinate system, which will eventually be sent to the sorting grasp node.
        (a, b) = (round(((320 - cx) / 4000), 5), round(((480 - cy) / 3000) *
0.8+0.15, 5))
        # Calculate joint 5 angle reference value based on corner offset angle
        vx = self.corners[0][0][0] - self.corners[1][0][0]
        vy = self.corners[0][0][1] - self.corners[1][0][1]
        target_joint5 = compute_joint5(vx,vy)
        print("target_joint5: ",target_joint5)
        if self.pub_pos_flag == True:
            # self.pubPos_flag value is False, indicating position topic publishing is prohibited to prevent sending recognized position messages again during grasping process
            self.pub_pos_flag = False
            # Create joint 5 angle reference value topic message object
            joint5 = Int16()
            joint5.data = int(target_joint5)
            # Publish joint 5 angle reference value topic
```

```
            self.pub_joint5.publish(joint5)
            # Create position information topic message object
            pos = AprilTagInfo()
            # pos.id represents target color block color, 1-4 represent red,
green, blue, yellow respectively
            pos.id = self.target_color
            pos.x = float(b)
            pos.y = float(a)
            pos.z = 0.03
            # Publish position information topic
            self.pos_info_pub.publish(pos)
```

process function:

```
def process(self,rgb_img,key):
    rgb_img = cv.resize(rgb_img, (640, 480))
    binary = []
    # If c or C is pressed, start using mouse to select an area of the target
color block for color calibration, release mouse to complete color calibration
    if key == ord('c') or key == ord('C'):
        self.target_color = 0
        self.Reset()
        self.updata_flag = True
    # If r or R is pressed, indicating target color block color is red, assign
self.target_color to 1
    elif key == ord('r') or key == ord('R'):
        self.target_color = 1
        self.cur_target_color = self.target_color
    elif key == ord('g') or key == ord('G'):
        self.target_color = 2
        self.cur_target_color = self.target_color
    elif key == ord('b') or key == ord('B'):
        self.target_color = 3
        self.cur_target_color = self.target_color
    elif key == ord('y') or key == ord('Y'):
        self.target_color = 4
        self.cur_target_color = self.target_color
    elif key == ord('i') or key == ord('I') or self.target_color!=0:
self.Track_state = "identify"
    # Check self.Track_state value. If it's init, it means no color is selected,
use mouse for color calibration
    if self.Track_state == 'init':
        cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)
        cv.setMouseCallback(self.windows_name, self.onMouse, 0)
        if self.select_flags == True:
            cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
            cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)
            if self.Roi_init[0] != self.Roi_init[2] and self.Roi_init[1] !=
self.Roi_init[3]:
                rgb_img, self.hsv_range = self.color.Roi_hsv(rgb_img,
self.Roi_init)
                self.dyn_update = True
            else: self.Track_state = 'init'
```

```
    # If self.Track_state value is "identify", it means entering recognition
mode. Then read HSV values of specified color based on self.target_color value
and assign to self.hsv_range. This value will be passed to color recognition
program later. Also assign self.cur_color and self.text_color, the former
represents currently recognized color and will be printed on the image for
display, the latter represents the image text color.
    elif self.Track_state == "identify":
        if self.target_color == 1:
            self.hsv_range = read_HSV(self.red_hsv_text)
            self.cur_color = "red"
            self.text_color = (0, 0, 255)


        elif self.target_color == 2:
            self.hsv_range = read_HSV(self.green_hsv_text)
            self.cur_color = "green"
            self.text_color = (0, 255, 0)


        elif self.target_color == 3:
            self.hsv_range = read_HSV(self.blue_hsv_text)
            self.cur_color = "blue"
            self.text_color = (255, 0, 0)

        elif self.target_color == 4:
            self.hsv_range = read_HSV(self.yellow_hsv_text)
            self.cur_color = "yellow"
            self.text_color = (255, 255, 0)

        else:
            self.Track_state = 'init'
    # Check if self.Track_state is not 'init', indicating entering color
recognition mode
    if self.Track_state != 'init':
        if len(self.hsv_range) != 0:
            # Pass image and self.hsv_range values to the object_follow function
of self.color object created during initialization, get color recognition result
image and color recognition results self.circle and corner information
self.corners
            rgb_img, binary, self.circle,_,self.corners=
self.color.object_follow(rgb_img, self.hsv_range)
            # Get enclosing circle center coordinates, which are the color block
center point coordinates
            self.cx = self.circle[0]
            self.cy = self.circle[1]
            # Get enclosing circle radius
            self.circle_r = self.circle[2]
            # When entering calibration mode (self.updata_flag value is True)
and after calibration ends, write hsv_range values to the HSV file of the
currently recognized color
            if self.cur_target_color == 1 and self.updata_flag == True:
                write_HSV(self.red_hsv_text, self.hsv_range)
            elif self.cur_target_color  == 2 and self.updata_flag == True:
                write_HSV(self.green_hsv_text, self.hsv_range)
            elif self.cur_target_color == 3 and self.updata_flag == True:
                write_HSV(self.blue_hsv_text, self.hsv_range)
            elif self.cur_target_color == 4 and self.updata_flag == True:
                write_HSV(self.yellow_hsv_text, self.hsv_range)
```

```
            self.updata_flag = False
    # Draw current recognized color information on the original color image

    rgb_img = cv2.putText(rgb_img, self.cur_color, (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, self.text_color, 2)
    return rgb_img, binary
```

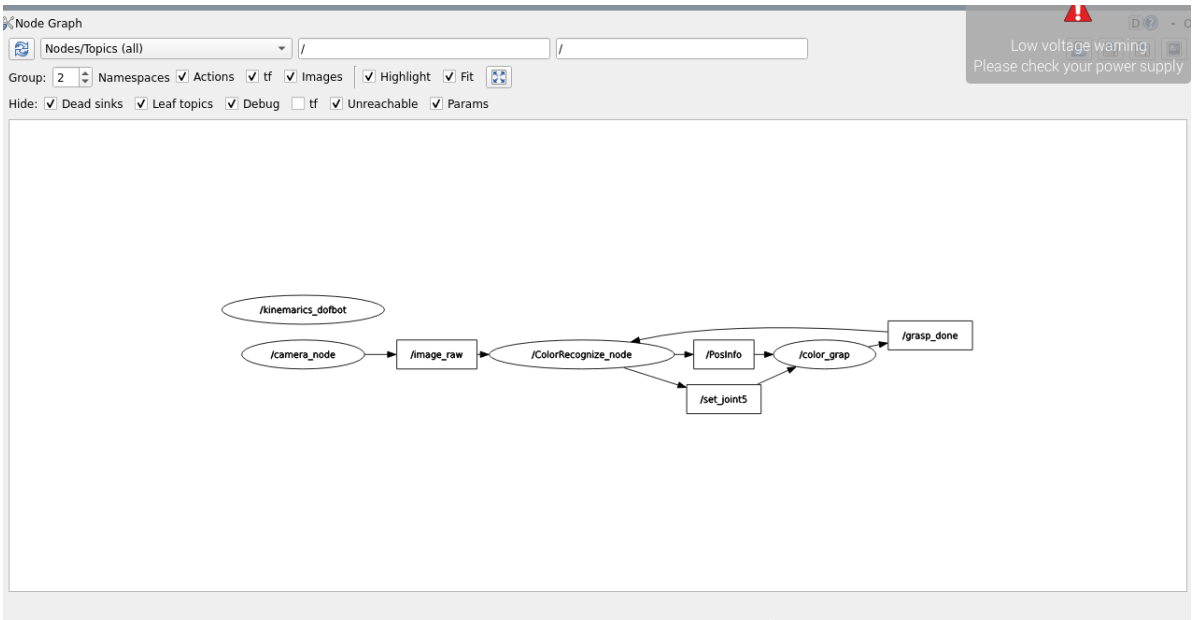
object_follow function, located at /root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/color_common.py:

```
def object_follow(self, img, hsv_msg):
    src = img.copy()
    # Create NumPy array from color range
    src = cv.cvtColor(src, cv.COLOR_BGR2HSV)
    lower = np.array(hsv_msg[0], dtype="uint8")
    upper = np.array(hsv_msg[1], dtype="uint8")
    # Create mask based on specific color range
    mask = cv.inRange(src, lower, upper)
    color_mask = cv.bitwise_and(src, src, mask=mask)
    # Convert image to grayscale
    gray_img = cv.cvtColor(color_mask, cv.COLOR_RGB2GRAY)
    # Get structural elements of different shapes
    kernel = cv.getStructuringElement(cv.MORPH_RECT, (5, 5))
    # Morphological closing operation
    gray_img = cv.morphologyEx(gray_img, cv.MORPH_CLOSE, kernel)
    # Image binary threshold operation
    ret, binary = cv.threshold(gray_img, 10, 255, cv.THRESH_BINARY)
    # Get contour point set (coordinates)
    find_contours = cv.findContours(binary, cv.RETR_EXTERNAL,
cv.CHAIN_APPROX_SIMPLE)
    if len(find_contours) == 3:
        contours = find_contours[1]
    else:
        contours = find_contours[0]
    if len(contours) != 0:
        areas = []
        for c in range(len(contours)): areas.append(cv.contourArea(contours[c]))
        max_id = areas.index(max(areas))
        max_rect = cv.minAreaRect(contours[max_id])
        self.max_box = cv.boxPoints(max_rect)
        self.max_box = np.int0(self.max_box)
        (color_x, color_y), color_radius = cv.minEnclosingCircle(self.max_box)
        self.Center_x = int(color_x)
        self.Center_y = int(color_y)
        self.Center_r = int(color_radius)
        perimeter = cv.arcLength(contours[max_id], True)
        self.approx = cv.approxPolyDP(contours[max_id], 0.035 * perimeter, True)
        cv.circle(img, (self.Center_x, self.Center_y), self.Center_r, (255, 0,
255), 2)
        cv.circle(img, (self.Center_x, self.Center_y), 2, (0, 0, 255), -1)
    else:
        self.Center_x = 0
        self.Center_y = 0
        self.Center_r = 0
    return img, binary, (self.Center_x, self.Center_y,
self.Center_r),self.max_box,self.approx
```

3.2. grasp

Already analyzed in [13.3D Sorting and Grasping Course]-[1. AprilTag ID Sorting]-[3. Code Analysis], you can refer to the content there.

4. Node Communication Diagram

After entering the docker container, enter the following commands in the docker terminal:

```
ros2 run rqt_graph rqt_graph
```